5W + 1H Selenium

Selenium adalah rangkaian perangkat lunak open-source untuk mengotomatisasi interaksi dengan browser. Dalam pengujian web, Selenium menjalankan tindakan seperti membuka halaman, mencari elemen, mengisi formulir, mengeklik tombol, dan memeriksa hasil seolah-olah dilakukan oleh pengguna.

1. WHAT Apa itu Selenium?

Selenium merupakan ekosistem alat dan pustaka untuk mengendalikan browser secara otomatis. Komponen yang paling sering digunakan untuk membuat skrip pengujian adalah Selenium WebDriver. WebDriver menyediakan antarmuka pemrograman agar kode dapat memberi perintah kepada browser melalui driver yang sesuai. Standar WebDriver ditetapkan oleh World Wide Web Consortium (W3C), sehingga pendekatannya tidak hanya terikat pada satu browser atau bahasa pemrograman.

Komponen utama Selenium:

- WebDriver: API untuk mengontrol browser melalui kode, misalnya Chrome, Firefox, Edge, atau Safari.
- Selenium IDE: ekstensi perekam dan pemutar ulang tindakan browser untuk membuat contoh otomasi tanpa banyak menulis kode.
- Selenium Grid: infrastruktur untuk menjalankan pengujian pada beberapa mesin, browser, atau sistem operasi.
- Selenium Manager: pengelola driver browser yang membantu menemukan dan menyiapkan driver yang cocok secara otomatis pada Selenium modern.

2. WHO Siapa yang menggunakan Selenium?

Selenium digunakan oleh orang-orang yang terlibat dalam pengembangan dan pengujian aplikasi web. Pengguna utamanya meliputi:

- Software Quality Assurance (QA) / Software Tester: menyusun skenario uji, menjalankan pengujian browser, dan melaporkan temuan.
- Automation Tester: menulis dan memelihara skrip otomatis, locator elemen, serta validasi hasil.
- Web Developer: menjalankan pengujian regresi untuk memastikan perubahan kode tidak merusak fitur yang sudah ada.
- DevOps / tim CI: mengintegrasikan pengujian browser ke pipeline agar aplikasi diperiksa secara berulang sebelum dirilis.

3. WHEN Kapan Selenium digunakan?

Selenium digunakan ketika perilaku aplikasi perlu diperiksa melalui browser, khususnya untuk skenario yang berulang atau harus dijalankan pada banyak kombinasi browser. Contohnya:

- Smoke testing: memeriksa fungsi penting setelah aplikasi selesai dibangun atau di-deploy.

- Regression testing: memastikan fitur lama tetap berjalan setelah ada perubahan kode.
- End-to-end testing: memeriksa alur dari awal hingga akhir, misalnya login, mencari produk, memasukkan ke keranjang, dan checkout.
- Cross-browser testing: memeriksa apakah alur yang sama berjalan pada Chrome, Firefox, dan Edge.

Catatan: Selenium berfokus pada otomasi browser. Selenium bukan alat utama untuk menguji logika fungsi secara terisolasi (unit testing), menguji API tanpa browser, atau mengukur beban/performa server.

4. WHERE Di mana Selenium digunakan?

Selenium dijalankan pada lingkungan yang memiliki browser dan runtime bahasa pemrograman. Untuk proyek Java, skrip biasanya dikembangkan di IDE seperti IntelliJ IDEA atau Eclipse, dikelola dependensinya dengan Maven/Gradle, lalu dijalankan pada komputer pengembang atau server pengujian.

- Lokal (local): pengembang menjalankan Chrome atau browser lain di komputernya untuk mencoba skenario.
- Staging / QA: pengujian dilakukan pada versi aplikasi yang mendekati lingkungan produksi sebelum dirilis.
- CI/CD: skrip berjalan otomatis pada pipeline, misalnya setelah perubahan kode atau sebelum rilis.
- Grid / remote: browser dijalankan pada mesin lain atau beberapa node untuk memperluas cakupan pengujian.

5. WHY Mengapa Selenium digunakan?

Selenium membantu mengurangi pekerjaan pengujian manual yang berulang dan membuat pemeriksaan alur browser lebih konsisten. Skrip yang sama dapat dijalankan berkali-kali setelah perubahan aplikasi, sehingga regresi lebih cepat ditemukan. Selenium juga bersifat open-source, mendukung beberapa bahasa pemrograman dan browser, serta dapat diintegrasikan dengan test runner dan pipeline pengembangan.

Hal yang perlu diperhatikan:

- Skrip perlu dipelihara ketika struktur halaman atau locator berubah.
- Pengujian dapat menjadi tidak stabil jika skrip tidak menunggu elemen siap. Gunakan explicit wait/ExpectedConditions, bukan mengandalkan jeda tetap secara berlebihan.
- Tidak semua pengujian layak diotomatisasi; pilih alur yang penting, sering diulang, dan memiliki hasil yang dapat diverifikasi.

6. HOW Bagaimana Selenium bekerja?

Secara umum, program Java mengirim perintah melalui WebDriver. WebDriver berkomunikasi dengan driver browser, lalu driver meneruskan perintah ke browser.

Browser menjalankan tindakan dan mengembalikan informasi halaman kepada skrip. Urutan kerja pengujian biasanya sebagai berikut:

1. Buat sesi browser: misalnya WebDriver driver = new ChromeDriver().
2. Buka halaman: driver.get(URL).
3. Temukan elemen: gunakan locator seperti By.id(), By.name(), atau By.cssSelector().
4. Lakukan tindakan: misalnya sendKeys() untuk mengetik dan click() untuk menekan tombol.
5. Tunggu kondisi halaman: gunakan WebDriverWait sampai elemen hasil terlihat atau kondisi tertentu terpenuhi.
6. Verifikasi hasil: ambil teks/atribut halaman dan bandingkan dengan hasil yang diharapkan.
7. Akhiri sesi: panggil driver.quit() agar browser dan proses WebDriver ditutup.

Contoh singkat (Java + Selenium WebDriver)

```
WebDriver driver = new ChromeDriver();
try {
    driver.get("https://www.selenium.dev/selenium/web/web-form.html");
    driver.findElement(By.name("my-text")).sendKeys("Uji Selenium");
    driver.findElement(By.cssSelector("button")).click();
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    String result = wait.until(ExpectedConditions.visibilityOfElementLocated(
        By.id("message"))).getText();
    System.out.println(result); // hasil yang diharapkan: Received!
} finally { driver.quit(); }
```

Kesimpulan: Selenium WebDriver mengotomatisasi tindakan pengguna pada browser untuk memeriksa apakah alur aplikasi web berjalan sesuai harapan. Kualitas skrip ditentukan oleh pemilihan skenario, locator yang tepat, penantian kondisi yang benar, dan verifikasi hasil.

Referensi resmi

- Selenium Documentation - Overview
- Selenium Documentation - WebDriver Getting Started
- Selenium Documentation - Waiting Strategies
- W3C - WebDriver Specification